9.5.4 Mixing patterns for PBAC

In practice, ACME often combines these patterns within a single policy. For example, this policy includes a relationship check (is the employee the owner's manager) and a discretionary membership verification (is the principal a member of the legal team).

```
permit(
    principal in ACME::Employee,
    action == ACME::"doc:edit",
    resource in ACME::Document
)
when {
    resource.owner.manager == principal
    || (principal in ACME::Team::"team:legal" &&
        resource.classification == "Legal")
};
```

This is where Cedar shines: the policy set can use attributes (ABAC), relationships (ReBAC), and team membership (RBAC) as needed. The entity store provides the facts, and Cedar evaluates them together in real-time.

9.6 Layering global constraints

Attribute-based access control (ABAC) is how ACME enforces organization-wide rules that span multiple products and policy patterns. Instead of duplicating checks in every application, the identity team writes a small set of global constraints that evaluate the request's context—things like device posture, risk level, or location—and allow those constraints to apply everywhere Cedar operates.

Here's a single global rule that stops employees from taking any document actions across Customer Collaboration when using unmanaged devices.